Blockchain
Smart Contract for Automated Supply Chain Management

Name: Diksha Bodkhe
Class: B.Tech - 2

Explaination:

1. What is this?

"This is a **smart contract** written in Solidity. A smart contract is like a digital agreement stored on blockchain that runs automatically.
Here We created a **Supply Manager Contract** that manages production and supply in a factory."

2. How does it work?

1. The factory **produces plastic bags** (measured in kilograms).
2. Whenever we add production in the contract, it **keeps track** of how much has been produced in total.
3. If production crosses **10,000 kg**, the contract automatically says:

'Supply needed!'
and it emits an **event** called SupplyRequested.

4. Then the supplier (can be anyone) delivers raw materials by calling markSupplyDelivered.
    - When supply is delivered, the contract **resets production** and confirms delivery with another event SupplyDelivered.

3. Why is this useful?

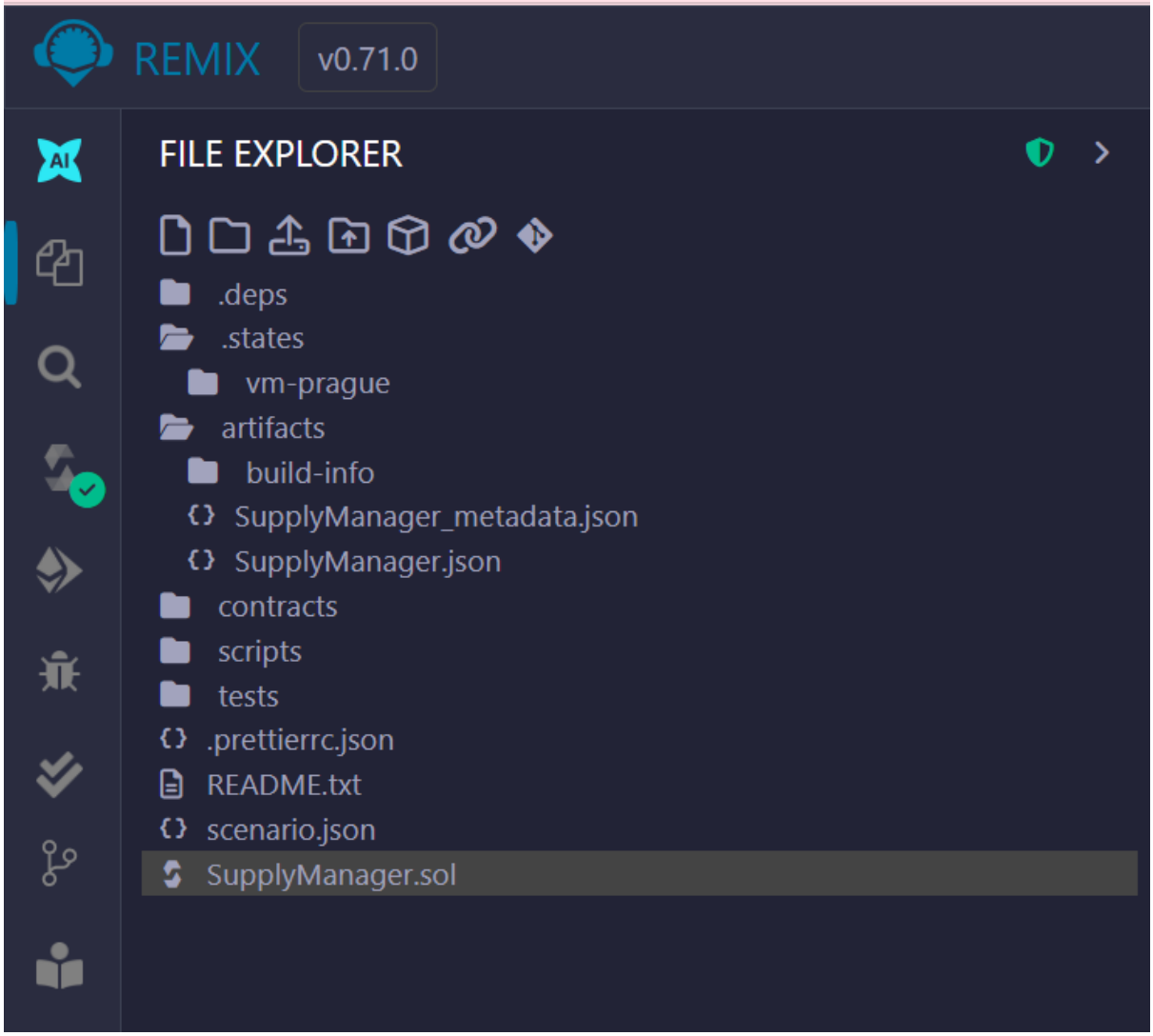
- In real life, managers often delay decisions (like when to order raw material).
- With blockchain + smart contract, the **decision is automatic** and **transparent**.
- No one can cheat or delay, because the rules are already coded in the contract.
- Everyone (factory manager + supplier) can see when supply is triggered and delivered.

4. What We did in Remix ?

- We wrote the contract in **Remix IDE** (free online Solidity editor).
- We compiled it with the Solidity compiler.

- We deployed it using **JavaScript VM** (a fake blockchain that runs in the browser).
- We tested by:
    - Adding 5000 kg production → nothing happens.
    - Adding another 6000 kg → threshold crossed → event SupplyRequested fired.
    - Delivering supply from another account → event SupplyDelivered fired → system reset.

Implementation:
Created SupplyManager.sol



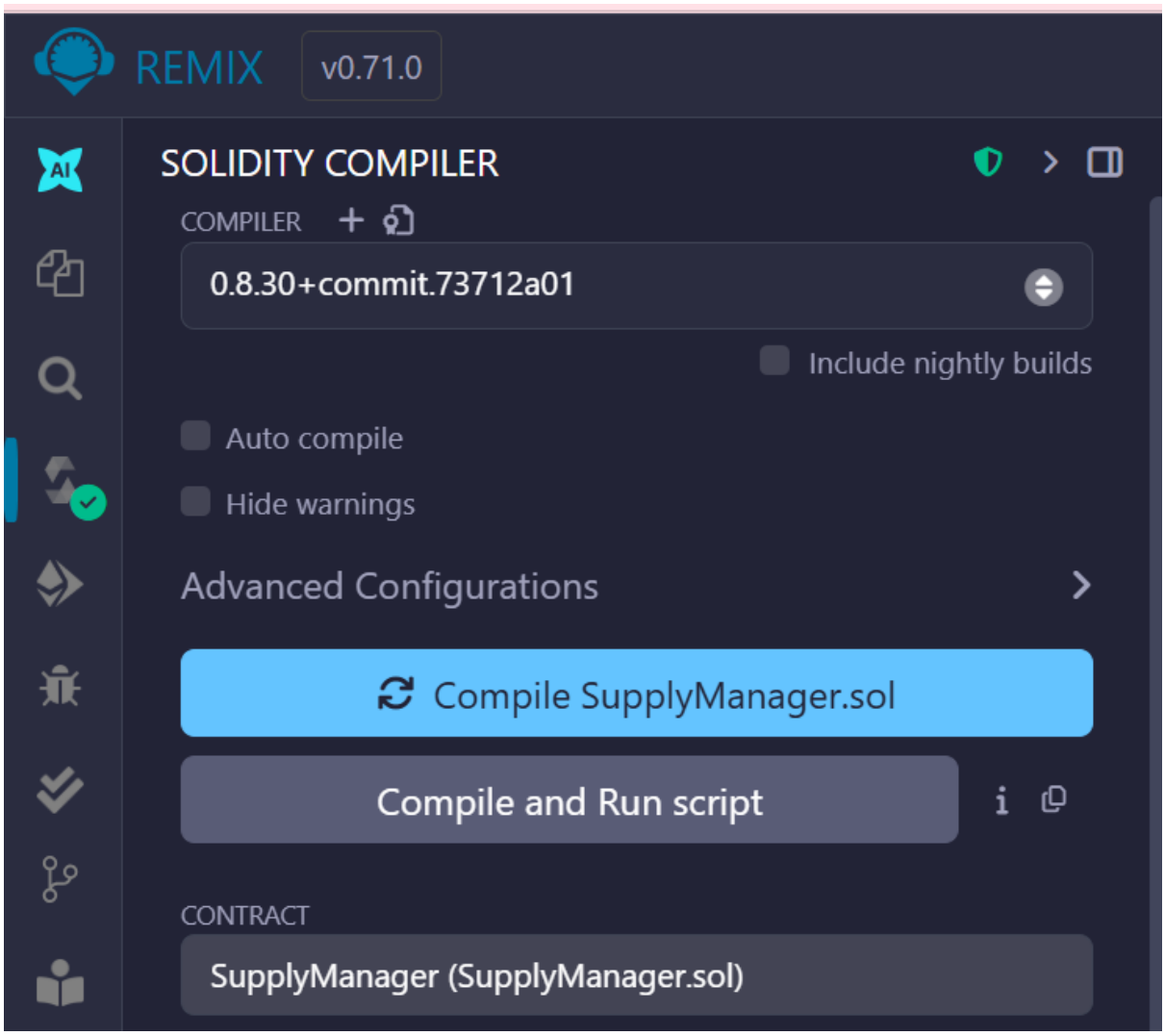
SupplyManager.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/// @title Simple Supply Manager for production -> supply trigger
/// @notice Production units are in kilograms (kg). 10 tonnes = 10,000 kg.
contract SupplyManager {
    address public owner;          // who deployed the contract (manager)
    uint256 public totalProducedKg;  // running total of production in kilograms
    uint256 public triggerKg;       // threshold to request supply (10,000 kg)
    bool public waitingForSupply;    // true if a supply request is active
```

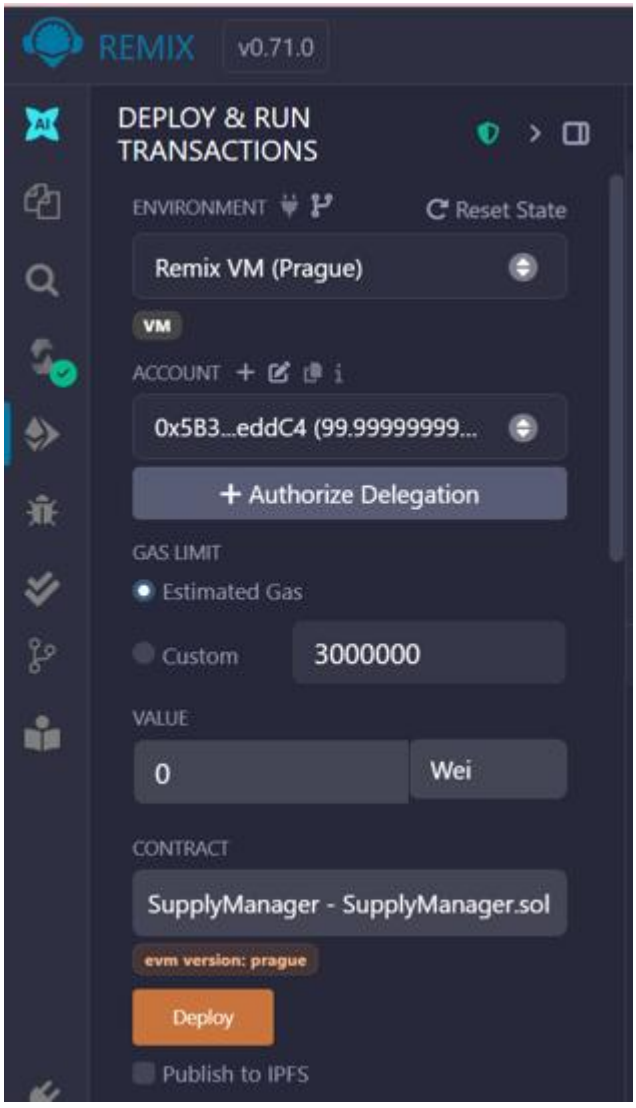
```
    // Event: emitted when supply is requested
    event SupplyRequested(uint256 requestedAmountKg, uint256 totalProducedKg);

    // Event: emitted when supply is marked delivered
    event SupplyDelivered(address supplier, uint256 deliveredAmountKg);

    modifier onlyOwner() {
        require(msg.sender == owner, "Only owner can call");
        _;
    }

    constructor() {
        owner = msg.sender;
        triggerKg = 10000; // 10 tonnes = 10,000 kg
        totalProducedKg = 0;
        waitingForSupply = false;
    }

    /// @notice Add production amount (kg). Anyone with permission can call.
    /// @param producedKg amount of production to add in kilograms
    function addProduction(uint256 producedKg) public {
        require(producedKg > 0, "Produce more than 0 kg");
        // increase total
        totalProducedKg += producedKg;

        // If we reached or exceeded the trigger and no active request -> request supply
        if (!waitingForSupply && totalProducedKg >= triggerKg) {
            waitingForSupply = true;
            // Decide requested amount: here we ask for a fixed restock amount (example:
20000 kg)
            uint256 requestedAmount = 20000; // example amount to request (in kg)
            emit SupplyRequested(requestedAmount, totalProducedKg);
        }
    }

    /// @notice Manager or supplier can mark that supply is delivered.
    /// @param deliveredKg amount delivered in kilograms
    function markSupplyDelivered(uint256 deliveredKg) public {
        require(waitingForSupply, "No supply request active");
```

```
        require(deliveredKg > 0, "Delivered must be > 0");

        // Optionally reset production counter or subtract used raw material.
        // For simple demo, we reset totalProducedKg back to 0 after delivery.
        totalProducedKg = 0;
        waitingForSupply = false;

        emit SupplyDelivered(msg.sender, deliveredKg);
    }

    /// @notice View function to see how close you are to threshold
    /// @return remainingKg amount of kg remaining to reach trigger (0 means
reached)
    function remainingToTrigger() public view returns (uint256) {
        if (totalProducedKg >= triggerKg) {
            return 0;
        } else {
            return triggerKg - totalProducedKg;
        }
    }
}
```

Open Solidity Compiler



Deploy on JavaScript VM

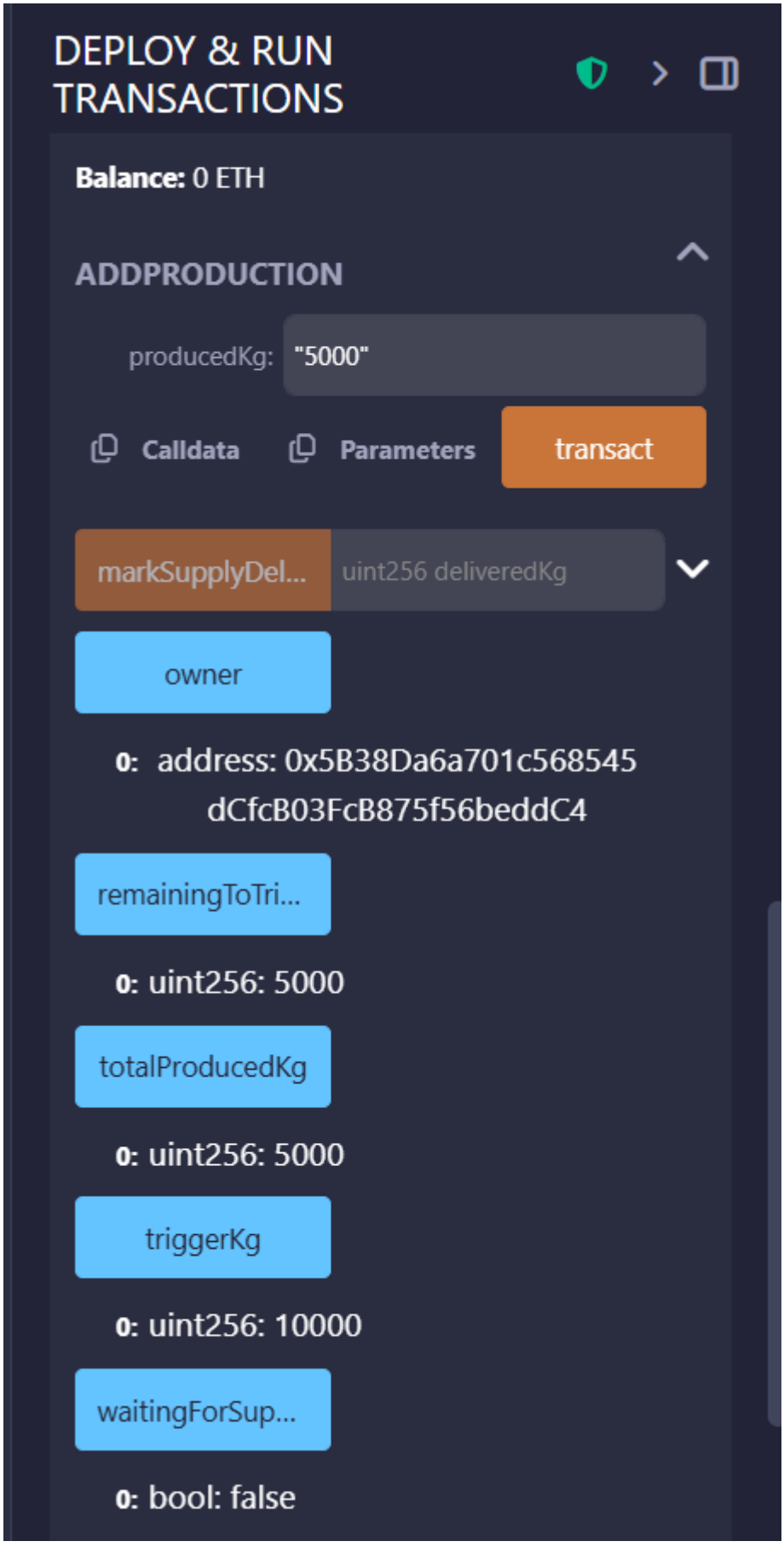


[vm] **from:** 0x5B3...eddC4 **to:** SupplyManager.(constructor) **value:** 0 wei **data:** 0x608...e0033 **logs:** 0 **hash:** 0x5d4...2531e Debug

status	0x1 Transaction mined and execution succeed
transaction hash	0x5d46696db79d24a37325cc78a86735ffda7aa90a011f5c430425a7ebea62531e
block hash	0xcd90d4ff69b2c903d0e8f371eaa1c757c8884346d8aedca0b3b530fc2add83ee
block number	8
contract address	0x9D7f74d0C41E726EC95884E0e97Fa6129e3b5E99
from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	SupplyManager.(constructor)
gas	560795 gas
transaction cost	487647 gas
execution cost	405947 gas

Deployed Contract

Simulate production that does NOT reach threshold



[vm]from: 0x5B3...eddC4to: SupplyManager.addProduction(uint256) 0xB57...9dF30value: 0 weidata: 0xaff...01388logs: 0hash: 0xab0...17183

status 0x1 Transaction mined and execution succeed

transaction hash 0xab078c1f0ccabae8bd5d4fdc44cd909e8b311d2c4a58a8c66fb0850fdec17183

block hash 0xce4f7dc93975b9274ad141cd397b835316cd6958d36daf37c5f5e5028da522d0

block number 39

from 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

to SupplyManager.addProduction(uint256) 0xB57ee0797C3fc0205714a577c02F7205bB89dF30

gas 55619 gas

transaction cost 48364 gas

execution cost 27148 gas

input 0xaff...01388

output 0x

decoded input {

"uint256 producedKg": "5000"

}

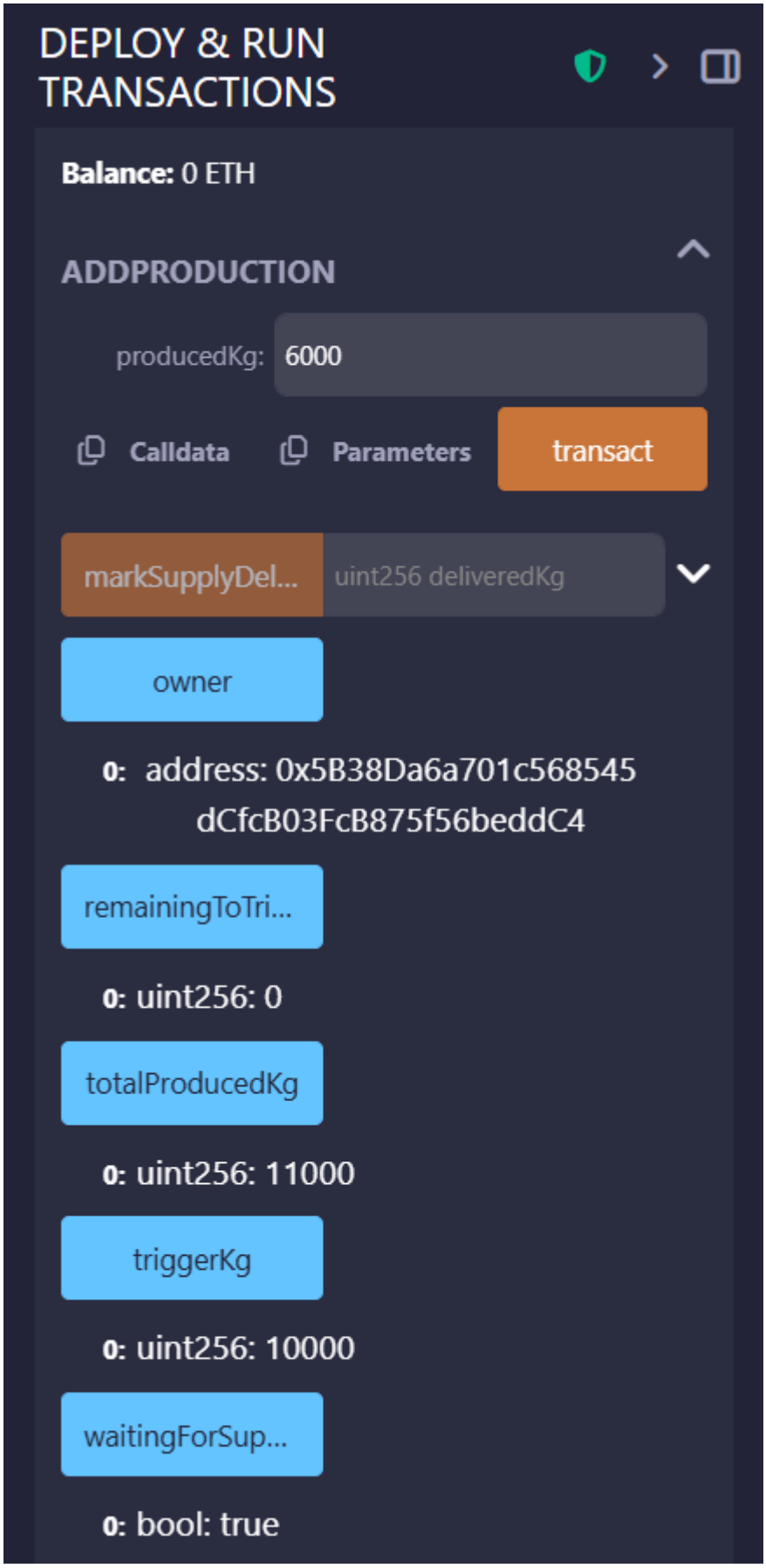
decoded output {}

logs []

raw logs []

transact to SupplyManager.addProduction pending ...

Simulate production reaching and exceeding threshold



[vm]from: 0x5B3...eddC4to: SupplyManager.addProduction(uint256) 0xB57...9dF30value: 0 weidata: 0xaff...01770logs: 1hash: 0x3b4...d25dd

status 0x1 Transaction mined and execution succeed

transaction hash 0x3b4145de3ca7e44004c61bd9bf3142baef7b5f6e8a6e50e38a5953ef6f2d25dd

block hash 0x3bdc4c442f87a73dff59d6ec888f0048653a89de0992088b66469fb43ad99359

block number 40

from 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

to SupplyManager.addProduction(uint256) 0xB57ee0797C3fc0205714a577c02F7205bB89dF30

gas 61052 gas

transaction cost 53088 gas

execution cost 31872 gas

input 0xaff...01770

output 0x

decoded input


```
{
"uint256 producedKg": "6000"
}
```


decoded output {}

logs


```
[
{
  "from": "0xB57ee0797C3fc0205714a577c02F7205bB89dF30",
  "topic": "0x721fcf0f1107b70eef501e79c852e3ed76a3eca3f8c3c7d350a3c5bfb5f11a5f",
  "event": "SupplyRequested",
  "args": {
        "0": "20000",
        "1": "11000"
  }
}
]
```

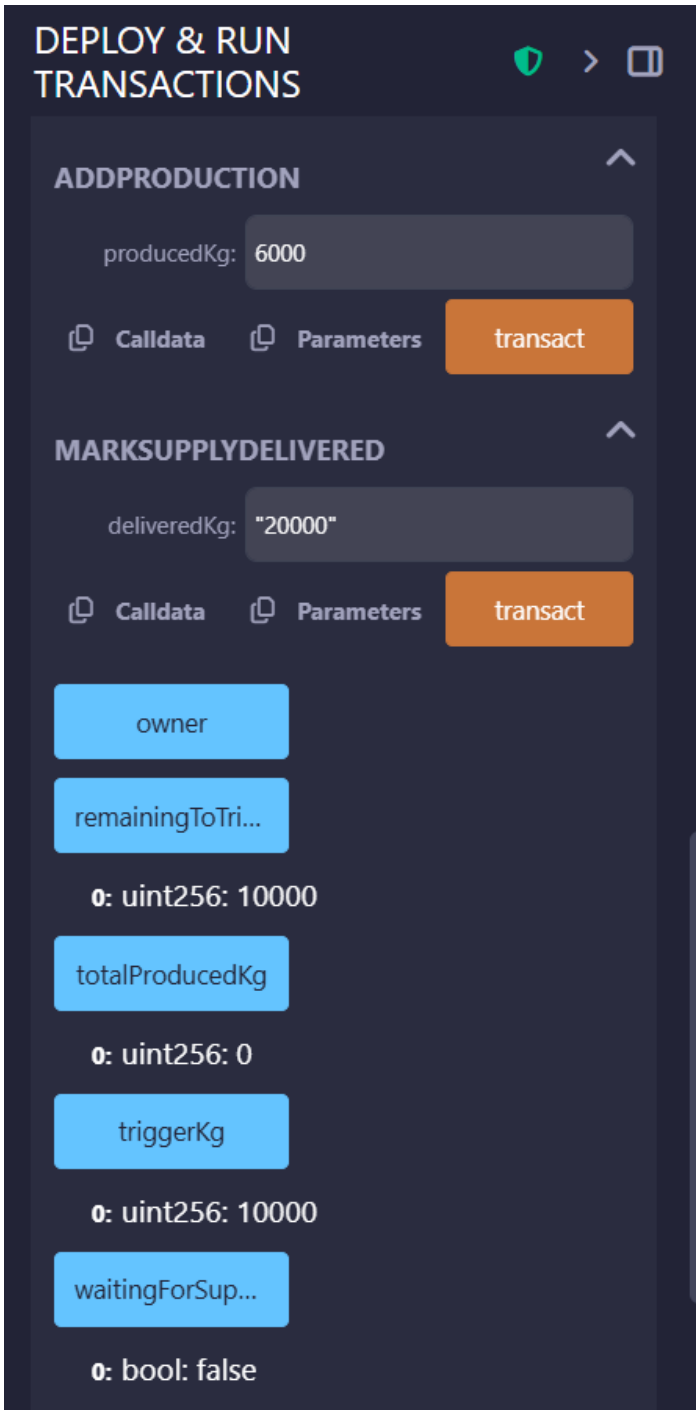

raw logs


```
[
 {
  "logIndex": "0x1",
  "blockNumber": "0x28",
  "blockHash": "0x3bdc4c442f87a73dff59d6ec888f0048653a89de0992088b66469fb43ad99359",
```

```
   "transactionHash":
"0x3b4145de3ca7e44004c61bd9bf3142baef7b5f6e8a6e50e38a5953ef6f2d25dd",
   "transactionIndex": "0x0",
   "address": "0xB57ee0797C3fc0205714a577c02F7205bB89dF30",
   "data":
"0x0000000000000000000000000000000000000000000000000000000000004e200
0000000000000000000000000000000000000000000000000000000002af8",
   "topics": [
    "0x721fcf0f1107b70eef501e79c852e3ed76a3eca3f8c3c7d350a3c5bfb5f11a5f"
   ]
  }
]
```

transact to SupplyManager.markSupplyDelivered pending ...

Simulate supplierdelivering supply



[vm]from: 0x5B3...eddC4to: SupplyManager.markSupplyDelivered(uint256) 0xB57...9dF30value: 0 weidata: 0x0ba...007d0logs: 1hash: 0xfc0...27bb9
status 0x1 Transaction mined and execution succeed
transaction hash
0xfc0758595376c888ba9194531ed2aaa8c52f600dbfca2df208631ee691f27bb9
block hash
0x675612eef00aaa5711740433f4bf0440ab8b794099c322c00eccb2fed7767f72
block number 41
from 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to SupplyManager.markSupplyDelivered(uint256) 0xB57ee0797C3fc0205714a577c02F7205bB89dF30
gas 49523 gas
transaction cost 26771 gas
execution cost 12247 gas
input 0x0ba...007d0
output 0x
decoded input {
"uint256 deliveredKg": "2000"
}
decoded output {}
logs [
{
"from": "0xB57ee0797C3fc0205714a577c02F7205bB89dF30",
"topic": "0x1ef8295688ca775a351d83f210ea4ffa6aa0c371106b8d57a301a9584ed5fcb3",
"event": "SupplyDelivered",
"args": {
"0": "0x5B38Da6a701c568545dCfcB03FcB875f56beddC4",
"1": "2000"
}
}
]
raw logs [
{
"logIndex": "0x1",
"blockNumber": "0x29",
"blockHash": "0x675612eef00aaa5711740433f4bf0440ab8b794099c322c00eccb2fed7767f72",
"transactionHash": "0xfc0758595376c888ba9194531ed2aaa8c52f600dbfca2df208631ee691f27bb9",

```
    "transactionIndex": "0x0",
    "address": "0xB57ee0797C3fc0205714a577c02F7205bB89dF30",
    "data":
"0x0000000000000000000000005b38da6a701c568545dcfcb03fcb875f56beddc4000
0000000000000000000000000000000000000000000000000000000000007d0",
    "topics": [
      "0x1ef8295688ca775a351d83f210ea4ffa6aa0c371106b8d57a301a9584ed5fcb3"
    ]
  }
]
```


Repeat: Continue calling addProduction to simulate production. Each time total crosses 10,000 kg and waitingForSupply is false, the contract will emit supply request again.